



Ask a Question



The Need For Micro Frontends

This lesson continues the discussion on micro frontends from the previous lesson.

We'll cover the following

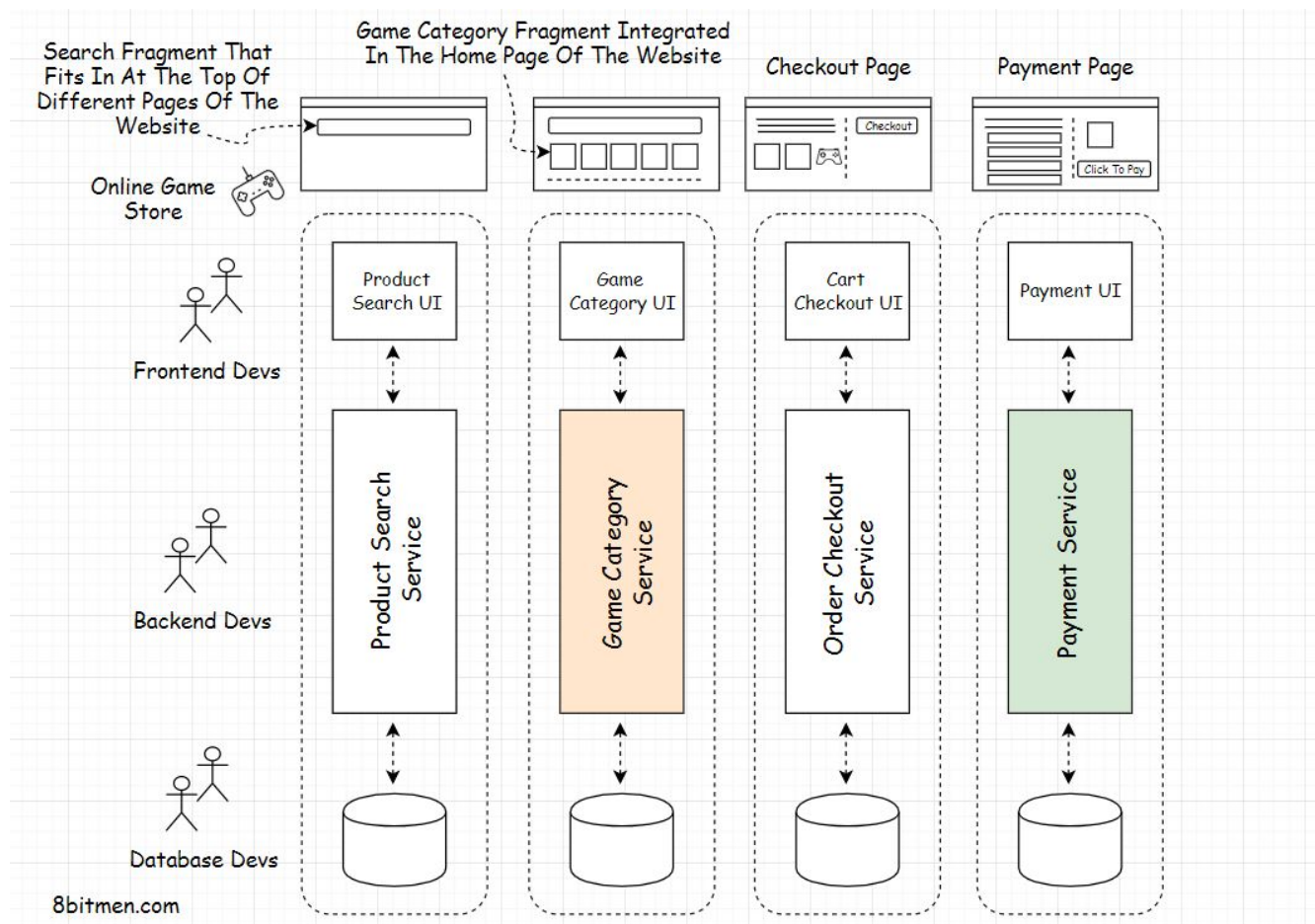


- Easier coordination between the frontend and the backend devs
- Leveraging the right technology

A micro frontend team may own a more extensive UI component like the *checkout page*. They may own a minor component that fits in a particular component like the *game category* component on the *home page*. Or they might own both.

The smaller components that integrate into other pages/components of the application are known as *fragments*.





Now, you might be wondering, what are the significant upsides of splitting an application into micro frontends?

Let's find out.

Easier coordination between the frontend and the backend devs

When we have full-stack teams owning an entire service end to end, this averts the need for a dedicated frontend team. In addition, since the frontend devs now work alongside the backend devs on the same team, this saves a lot of time that was initially spent in the cross-team coordination. ?

With this new approach, communication is quick and not so formal. This improves the team's productivity and enables them to deliver a better user experience by having more effective coordination between the backend and the frontend devs. ☺

Leveraging the right technology

Since the micro frontends are loosely coupled, we can develop them leveraging different technologies, just like microservices. This opens up our options as opposed to sticking to just one UI technology to build the complete front end of the website.

There are a plethora of existing frontend technologies in the industry that cater to different use cases, in addition to the new waves of *JavaScript* frameworks that hit us every year.

With the micro frontends approach, we can pick the right technology to build our frontend components. We often have use cases where just *plain JavaScript, HTML, and CSS* suffice to build a feature, and then there are other cases where we need advanced frameworks like *React, Angular, and Vue* to implement our features.

With micro frontends, we don't necessarily have to use *React* or a similar framework to build a certain component if we don't need it, despite having other components being implemented using it.

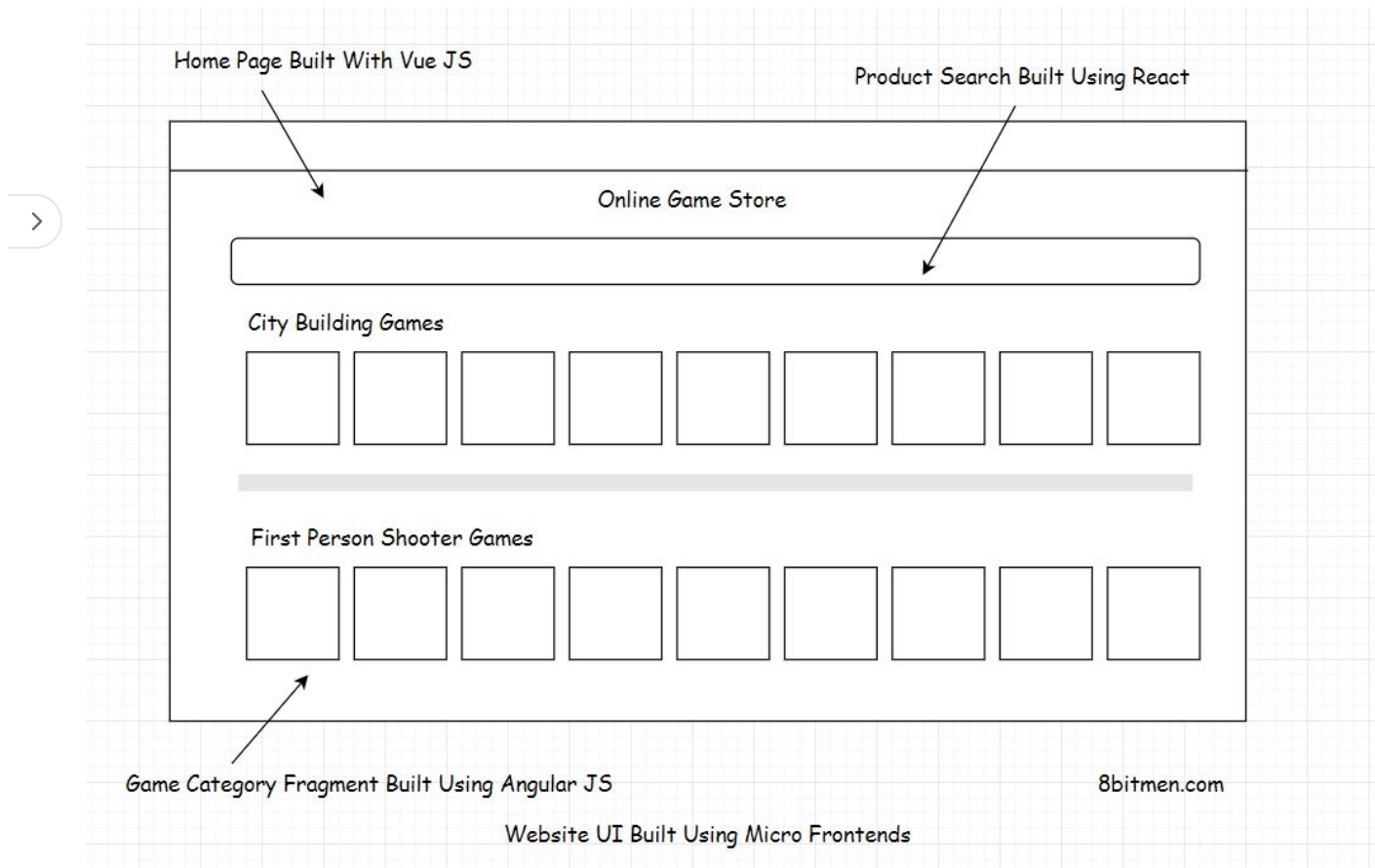
On the flip side, if our website is built on *plain JavaScript* and we want to use *React*, we don't have to re-write the entire website to use *React*. We can just write specific components that need *React* and integrate them into the website.

Even if multiple teams use the same technology to build their UI components, they can work on different versions of the technology. They can easily upgrade their libraries without impacting the website's other UI components.

?

Tr





Now, moving forward with the micro frontends approach may sound delightful, but it's only fit for medium to large websites. This approach won't be that advantageous for simple use cases. Rather, it will make things more complex and prove to be overkill.

Using multiple technologies in a project brings along a lot of architectural and maintenance complexities with it.

With micro frontends, we need to write additional code to combine all the components built with heterogeneous tech. We also have to deal with

?

Tr

